



UNIVERSITÀ DEGLI STUDI DI SALERNO

Corso di Fondamenti di Intelligenza Artificiale (FIA)

Anno Accademico 2025/26



HAIVE

an AI for the bug-themed strategy game

16 febbraio 2026

Autore	Matricola
Forte Guglielmo	0512119526
Rega Maristella	0512119032
Russo Serena	0512119098
Squitieri Andrea	0512119008

LINK GITHUB: <https://github.com/andrea-sq/hAIve>

Indice

1	Introduzione	2
2	Descrizione del problema	3
2.1	Obiettivi	3
2.2	Specifiche PEAS	3
2.2.1	Caratteristiche dell'ambiente	3
2.3	Analisi del problema	4
2.3.1	Rappresentazione dello stato di gioco	5
2.3.2	Contributi precedenti	5
3	Soluzioni proposte	6
3.1	Esplorazione mosse	6
3.1.1	Potatura alfa-beta	6
3.1.2	Monte Carlo Tree Search	8
3.2	Valutazione stato di gioco	9
3.2.1	Funzioni euristiche	9
3.2.2	NNUE	10
4	Confronto tra le implementazioni	12
4.1	Specifiche hardware	12
4.2	Implementazioni con funzione euristica	12
4.2.1	Negamax vs Random	12
4.2.2	MCTS vs Random	12
4.2.3	Negamax vs Young Brothers Wait	13
4.2.4	Negamax vs MTD(f)	13
4.2.5	MTD(f) vs MCTS	13
4.2.6	MTD(f) vs MzingaEngine	14
4.3	Implementazione di NNUE	15
4.3.1	Generazione dati sintetici	15
4.3.2	Addestramento della rete neurale	15

1 Introduzione

Hive è un gioco da tavolo strategico per due giocatori, ideato da John Yanni e pubblicato nella prima edizione nel 2001.

Rientra nei giochi deterministici ad informazione perfetta e a somma zero, tuttavia, a differenza di esempi più celebri come il gioco degli scacchi, non possiede una plancia e può essere svolto su qualsiasi superficie piana.

In seguito alla più recente espansione, il numero totale di pezzi è 28, di cui 14 bianchi e 14 neri, raffiguranti un insieme di insetti, aracnidi e isopodi, ognuno con un modo particolare di muoversi. L'obiettivo è catturare l'*ape regina* avversaria, facendo sì che sia completamente circondata da altri pezzi.

Perché Hive? Un'analisi del branching-factor [1] mostra che, negli stadi più avanzati del gioco, il fattore di ramificazione è stabile a 60. Pertanto, la crescita esponenziale di Hive è circa due volte quella degli scacchi, il cui fattore di ramificazione è intorno al 30.

L'interesse nel creare un'intelligenza artificiale in grado di giocare ad Hive non deriva solo dall'elevata complessità, ma anche dalla scarsa notorietà del gioco che lo rende un problema non ancora esplorato esaustivamente.

2 Descrizione del problema

2.1 Obiettivi

Lo scopo del progetto consiste nella realizzazione di un **modello di intelligenza artificiale** in grado di sfidare un altro agente intelligente — artificiale o umano — nel gioco di *Hive*.

2.2 Specifiche PEAS

- **Performance.** Le misure adottate per valutare le prestazioni sono il *tempo* per generare una mossa e le *percentuali di vittoria* contro una generatore di mosse randomiche e altri agenti noti.
- **Environment.** L'ambiente in cui opera l'agente è costituito dall'insieme di tutte le possibili configurazioni che l'alveare può assumere.
- **Actuators.** L'agente agisce sull'ambiente mediante la successiva mossa scelta.
- **Sensors.** L'agente percepisce l'ambiente mediante la sequenza di mosse dell'avversario.

2.2.1 Caratteristiche dell'ambiente

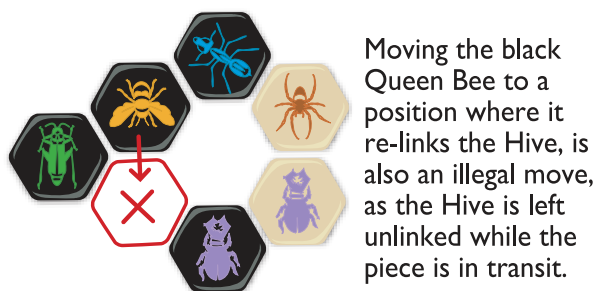
L'ambiente operativo è:

- **Completamente osservabile.** I sensori forniscono accesso allo stato completo dell'ambiente in ogni momento.
- **Deterministico.** Lo stato successivo è completamente determinato da stato corrente e dalla seguente azione scelta.
- **Sequenziale.** Ogni decisione può influenzare tutte quelle successive.
- **Statico.** L'ambiente rimane invariato mentre l'agente delibera.
- **Discreto.** Il numero di percezioni e azioni distinte è finito.
- **Multi-agente competitivo.** L'ambiente consente la presenza di esattamente due agenti, ognuno dei quali tenta di sconfiggere l'avversario.

2.3 Analisi del problema

Il gioco è costituito da poche e semplici regole, di seguito riassunte per facilitare la comprensibilità del problema.

- Ad ogni turno un giocatore potrà eseguire una delle due possibili mosse:
 1. Far entrare in gioco un insetto (finché ne restano a disposizione);
 2. Muovere un insetto già posizionato (se in grado di farlo).
- Facendo entrare in gioco un insetto, lo si deve posizionare in modo che nessuno dei suoi lati tocchi un insetto del colore avversario già posizionato, ma unicamente insetti del proprio colore. (Fa eccezione la prima mossa di ogni giocatore).
- L'ape regina deve entrare in gioco entro il quarto turno e, in mancanza di essa, i pezzi in gioco non possono muoversi.
- Se un giocatore non può muovere un suo pezzo in campo né aggiungerne di nuovi, passa il turno all'avversario: il gioco continua in questo modo finché il primo giocatore non è nuovamente in grado di fare la sua mossa.
- *One Hive Rule*: i pezzi in gioco devono essere costantemente connessi, non è mai possibile dividere l'alveare in due o più parti.



- *Freedom To Move*: poiché le creature si muovono scorrendo intorno all'alveare, un pezzo può muoversi da o verso una posizione solo se può fisicamente scivolare in quella direzione. Fa eccezione a questa regola il Grillo, ed eventualmente la Zanzara quando lo imita, in quanto può saltare sopra l'alveare. La Coccinella e lo Scarabeo sono in grado di spostarsi da o verso posizioni totalmente circondate muovendosi sopra l'alveare, ma devono comunque rispettare la regola nel caso al secondo livello vi sia un "varco stretto".



2.3.1 Rappresentazione dello stato di gioco

Per modellare lo stato in cui si trova l'alveare sono state adottate due rappresentazioni distinte:

- Una mappa esagonale modulare di raggio 14 (e diametro 29) a cui abbiamo applicato le coordinate assiali per identificare univocamente la posizione di ogni esagono. La scelta del raggio corrisponde al più piccolo numero che permette di posizionare tutti i 28 pezzi e mantenere l'illusione di una mappa infinita, in modo da minimizzare la quantità di spazio necessario a memorizzare ogni stato.
- Un vettore binario che rappresenta la presenza o l'assenza di possibili archi e l'altezza dei pezzi che sono in grado di salire sull'alveare.

2.3.2 Contributi precedenti

Il progetto Mzinga [2] è una collezione di software open-source per agevolare la comunità di sviluppatori e giocatori di Hive.

Offre tra le principali feature l'**Universal Hive Protocol** (UHP) che supporta l'interoperabilità tra software dedicati al gioco. Opera distinguendo tra due principali tipi di programmi: engines e viewers. Un engine ha il compito di comprendere come funziona una partita di Hive. Un viewer semplifica l'interazione dell'utente umano con l'engine.

Abbiamo deciso di implementare **UHP** per semplificare l'interazione e il confronto tra il nostro agente e altri agenti pre-esistenti. Ulteriore beneficio di questa scelta è derivato dall'utilizzo del viewer di Mzinga, che ci ha fornito un'interfaccia di gioco ampiamente utilizzata dalla community.

3 Soluzioni proposte

A partire dall'analisi effettuata, risulta naturale che si tratti di un problema di ricerca con avversario.

Per giungere ad una soluzione è necessario definire due componenti:

- **Algoritmo di esplorazione delle mosse.** Per ricercare la scelta più conveniente, non è sufficiente conoscere le mosse possibili in un dato istante, ma è necessario studiare le implicazioni di queste e quindi le possibili mosse n turni nel futuro.
- **Algoritmo di valutazione dello stato di gioco.** È impensabile esplorare completamente l'albero di gioco a causa dell'elevato *fattore di ramificazione*, che ricordiamo essere pari a 60. Di conseguenza, è necessario poter usufruire di una nozione della bontà di un determinato stato di gioco.

3.1 Esplorazione mosse

3.1.1 Potatura alfa-beta

La potatura alfa-beta è un algoritmo di ricerca che ha l'obiettivo di ridurre il numero di nodi esplorati dall'algoritmo minimax. Interrompe l'esplorazione di un nodo nel momento in cui determina che la mossa in oggetto sia sicuramente peggiore di almeno un'altra mossa precedentemente esplorata. Non è pertanto necessario continuare a esplorare tali mosse.

Negamax

La ricerca **negamax** è una variante dell'algoritmo minimax che sfrutta la proprietà di *somma zero* di giochi come Hive. Semplifica l'implementazione dell'algoritmo minimax basandosi sulla seguente proprietà: $\min(a, b) = -\max(-b, -a)$. Ciò implica che il valore di una posizione del giocatore A è la negazione del valore del giocatore B, che permette di non dover dividere il processo di ricerca in due casi distinti.

Algoritmo Negamax

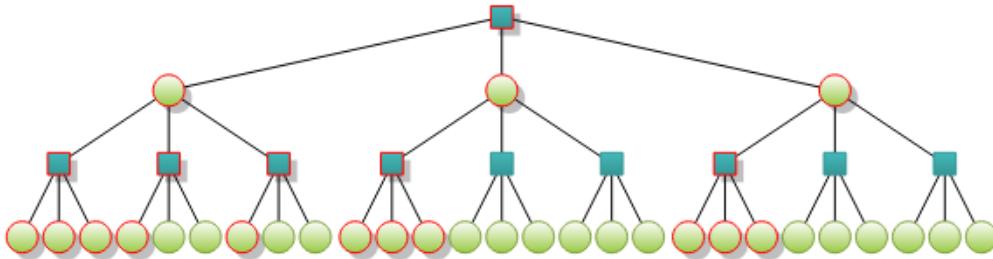
```
function negamax(node, depth,  $\alpha$ ,  $\beta$ , color) is
  if depth = 0 or node is a terminal node then
    return color * the heuristic value of node

  childNodes := generateMoves(node)
  childNodes := orderMoves(childNodes)
  value :=  $-\infty$ 
  foreach child in childNodes do
    value := max(value, -negamax(child, depth - 1,  $-\beta$ ,  $-\alpha$ , -color))
     $\alpha$  := max( $\alpha$ , value)
    if  $\alpha \geq \beta$  then
      break (* cut-off *)
  return value
```

Concetto Young Brothers Wait

Il **concetto Young Brothers Wait** (YBWC) è un elemento fondamentale per la ricerca alfa-beta parallela, coniato da Rainer Feldmann et al. nel 1986 [3].

I fratelli sono nodi che condividono lo stesso nodo padre. Il fratello maggiore, ovvero il primo rispetto ad una determinata funzione d'ordine, è esplorato per primo. I fratelli minori, prima di poter essere esplorati in parallelo, devono attendere che il fratello maggiore ceda il controllo a seguito di un taglio beta nella speranza che riduca i limiti dell'intervallo $[\alpha, \beta]$.



In the best-case alpha-beta game tree, the degree of every node is either 1 or b .

Young siblings wait [FMMV89]

Spawn the first child and immediately sync.

If beta-cutoff, done.

Otherwise, spawn all other children in parallel.

Pray that you don't waste any work.

Figura 1: Slide estratta dalla lezione di M. Frigo "The Cilk system for multi-threading programming"

MTD(f)

MTD(f) è un algoritmo di ricerca alfa-beta con finestra nulla e con tabella di trasposizione per memorizzare i risultati intermedi, creato da Aske Plaatt. Il nome è un'abbreviazione di MTD(n, f) ovvero **Memory-enhanced Test Driver** con nodo n e valore f .

L'efficacia di questo paradigma dipende da una buona stima iniziale del valore finale minimax. In media, migliore è questa supposizione più efficiente sarà l'algoritmo, poiché dovrà svolgere meno passi prima di convergere al valore corretto. Se la stima corrisponde al valore minimax, eseguirà solo due passi: il primo necessario ad individuare il limite superiore del valore x e il secondo per individuare il limite inferiore dello stesso valore.

All'interno del nostro motore di gioco viene utilizzato come stima il precedente valore di valutazione della board della precedente mossa, ad eccezione della prima mossa in cui la stima viene inizializzata a 0.

Algoritmo MTD(f)

```

function MTD(f, d) is
  g := f
  upperBound := +∞
  lowerBound := -∞

  while lowerBound < upperBound do
    β := max(g, lowerBound + 1)
    g := AlphaBetaWithMemory(root, β - 1, β, d)
    if g < β then
      upperBound := g
    else
      lowerBound := g

  return g

```

3.1.2 Monte Carlo Tree Search

MCTS è un algoritmo di **ricerca euristica** utilizzato per l'emulazione di processi decisionali, specialmente all'interno di software in grado di risolvere giochi da tavolo. Nel 2016 è stata per la prima volta dimostrata l'efficacia di MCTS combinato con le reti neurali.

L'algoritmo è basato su esplorazioni randomiche dello spazio di ricerca, a partire dalle quali costruisce gradualmente un albero di gioco e migliora la propria accuratezza.

Le quattro fasi

- **Selezione.** L'albero è esplorato dalla radice R fino a raggiungere una foglia L , dove la radice è lo stato di gioco corrente e una foglia è qualsiasi nodo che ha figli non ancora esaminati.
- **Espansione.** A meno che L non termini il gioco in maniera decisiva, viene espanso un nodo figlio C .
- **Simulazione.** Effettua una partita casuale a partire dal nodo C .
- **Backpropagation.** Utilizza i risultati della partita per aggiornare le informazioni nei nodi sul percorso da C a R .

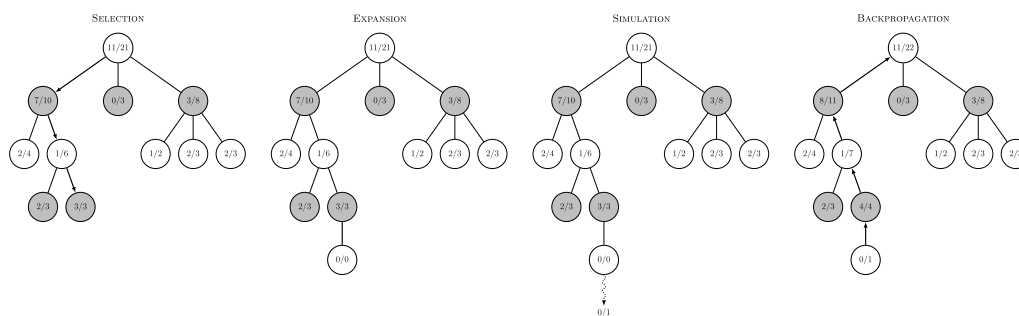


Figura 2: I 4 step della ricerca ad albero di Monte Carlo

3.2 Valutazione stato di gioco

3.2.1 Funzioni euristiche

L'obiettivo è la definizione di una funzione che restituisca una valutazione della bontà (di qui in avanti *score*) del corrente stato dell'alveare. Abbiamo deciso di implementarla mediante la composizione dei seguenti fattori:

- **Valore dei pezzi.** A ogni pezzo è assegnato un valore.
 - **Ape Regina:** 12
 - **Zanzara:** 8
 - **Formica:** 7
 - **Scarabeo:** 6
 - **Coccinella:** 6
 - **Onisco:** 5
 - **Cavalletta:** 3
 - **Ragno:** 2
- **Pezzi non in gioco.** Per ogni diversa creatura viene calcolata la differenza tra i pezzi non in gioco propri e dell'avversario. Questa viene poi moltiplicata per il valore della creatura e il fattore di pezzo non giocato. Viene premiato l'averne meno pezzi in gioco rispetto all'avversario.
- **Pezzi mobili in gioco.** Lo score viene calcolato moltiplicando il valore dei pezzi per il fattore di mobilità delle creature. Inoltre, il valore dei pezzi disposti sopra l'alveare è doppio.
- **Libertà dell'Ape Regina.** Per ogni pezzo adiacente alla propria Ape Regina viene rimosso dallo score il fattore di libertà e, in maniera inversa, per ogni pezzo adiacente all'Ape Regina nemica il fattore viene sommato allo score.
- **Versatilità della Zanzara.** Quando è in gioco, il valore della Zanzara corrisponde alla somma tra il numero di pezzi adiacenti ad essa e il valore massimo tra le creature adiacenti, fatta eccezione per le Api Regina e l'altra Zanzara.
- **Onisco's Law.** Se il proprio Onisco è in grado di spostare la propria Ape Regina in una posizione tale che vi siano più di due spazi liberi adiacenti ad essa, allo score viene sommato il fattore di difesa della Regina. In modo inverso, se l'Onisco avversario è in grado di effettuare la stessa mossa con l'Ape Regina avversaria, il fattore è sottratto allo score. Inoltre, se è possibile piazzare un Onisco vicino all'Ape Regina viene sommato (o, nel caso inverso, sottratto) metà del fattore di difesa allo score.

3.2.2 NNUE

NNUE, spesso rappresentata come $\mathbb{EUNN}$, è l'acronimo di **Efficiently Updatable Neural Networks**.

Viene utilizzata in giochi strategici virtuali, come *Shogi* o *scacchi*, come funzione di valutazione basata su rete neurale i cui input sono array di valori booleani che identificano una specifica combinazione (pezzo, cella). Fa affidamento sulla limitatezza della variazione di stato ad ogni mossa, tipico di questa categoria di giochi.

La rete è stata inventata da Yu Nasu ed introdotta all'interno di Shogi online nel 2018. Nell'agosto 2020 fu per la prima volta utilizzata all'interno di un software di scacchi, Stockfish 12, risultando in un incremento importante della forza di gioco dell'engine.

Le NNUE sono progettate affinché possano essere eseguite efficientemente su CPU, utilizzando tecniche di computazione incrementale e sfruttando l'architettura SIMD. Questo va in contrapposizione ai motori scacchistici basati su reti neurali profonde, come *Leela Chess Zero*, che necessitano di una GPU per inferire.



Figura 3: L'acronimo è un gioco di parole che fa riferimento alla figura mitologica giapponese *Nue*

Architettura

Nella sua forma più semplice, una rete NNUE è composta da tre livelli:

- **Input layer.** La dimensione nella nostra rappresentazione di Hive è pari a 17704 neuroni. Questo numero corrisponde alla somma dei prodotti tra numero di celle presenti nella mappa esagonale * numero totale di pezzi e tra le sei altezze al di sopra dell'alveare * i sei diversi pezzi in grado di salire sull'alveare.
- **Hidden layer.** Generalmente la dimensione di questo strato è arbitraria. Nonostante ciò è usuale utilizzare potenze di due, particolarmente nel range [256, 1024].
- **Output layer.** Questo livello corrisponde alla predizione del modello ed è perciò costituito da un solo neurone.

Perspective

La maggior parte delle architetture NNUE gestisce separatamente le prospettive degli avversari mediante due accumulatori distinti. Entrambe le prospettive usano pesi e bias di input identici. Durante il processo di inferenza, i due accumulatori sono concatenati e ordinati in modo alternato in base al turno di gioco. Il vettore risultante, di dimensione doppia rispetto ai singoli accumulatori, è il "vero" *hidden layer* ed è ciò che viene inoltrato all'*output layer*.

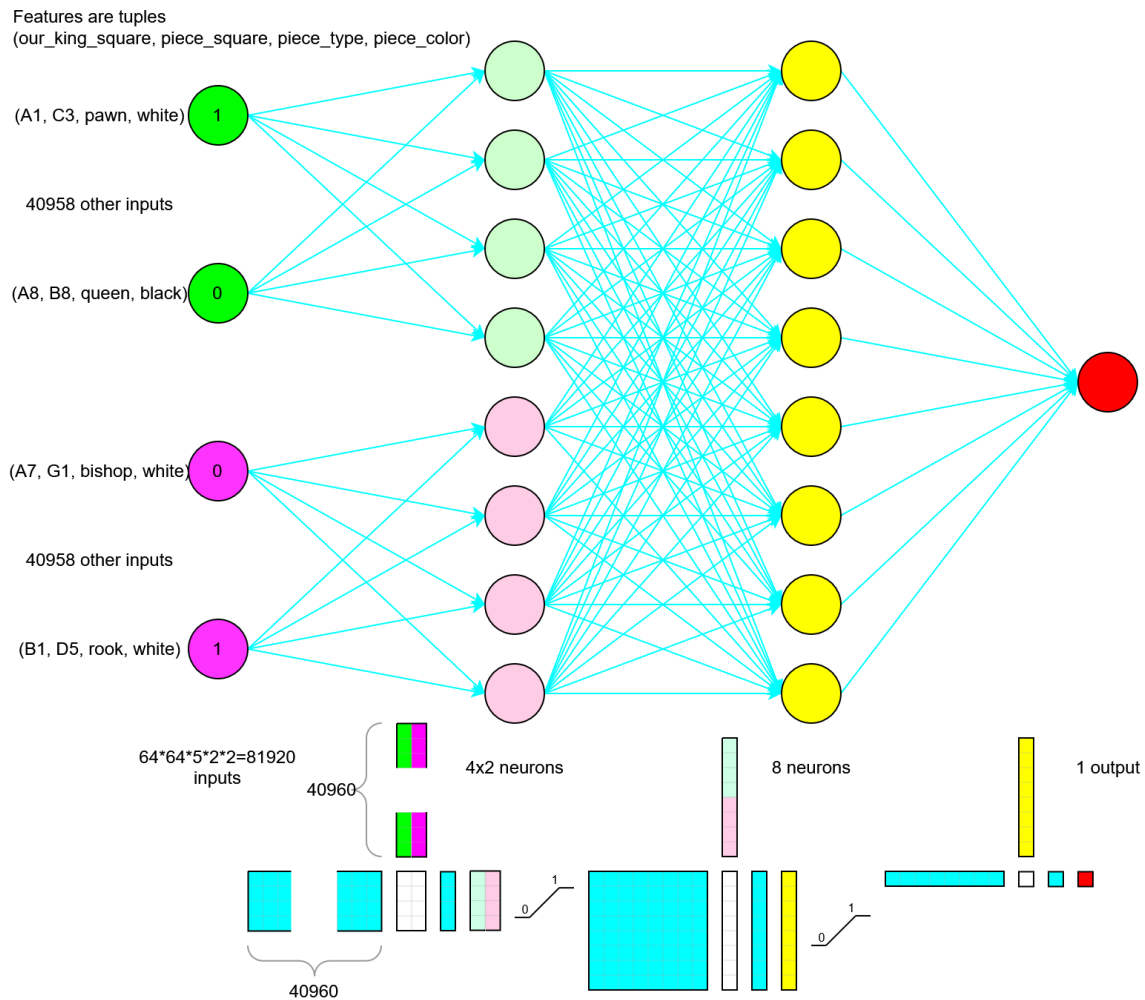


Figura 4: Esempio di architettura NNUE per il gioco degli scacchi

4 Confronto tra le implementazioni

4.1 Specifiche hardware

Tutte le misurazioni sono eseguite su una macchina avente le seguenti specifiche:

- **CPU:** Intel Core i7-14700HX 5.5 GHz
- **Memoria:** 32 GiB di RAM

4.2 Implementazioni con funzione euristica

4.2.1 Negamax vs Random

Questo confronto fornisce una valutazione della competitività del nostro agente Negamax contro un agente che effettua mosse randomiche (di qui in avanti *Random*), in modo tale da verificare la validità dell'agente implementato.

La valutazione è effettuata in base alla percentuale di vittorie su un campione di 1000 partite, considerando un tempo massimo per mossa di un secondo (1s) e alternando quale agente sia il primo giocatore.

Tipologia	Negamax	Random	Pareggi
Totale delle partite	100%	0%	0%
Negamax bianco (effettua la prima mossa)	100%	0%	0%
Negamax nero (effettua la seconda mossa)	100%	0%	0%

4.2.2 MCTS vs Random

Questo confronto fornisce una valutazione della competitività del nostro agente Monte Carlo Tree Search contro un agente *Random*, in modo tale da verificare la validità dell'agente implementato.

La valutazione è effettuata in base alla percentuale di vittorie su un campione di 100 partite, considerando un tempo massimo per mossa di un secondo (1s) e alternando quale agente sia il primo giocatore. All'agente MCTS sono stati forniti 5 rollout prima di espandere un nodo figlio ed è stata imposta una profondità massima di 200 nodi per rollout.

Tipologia	MCTS	Random	Pareggi
Totale delle partite	100%	0%	0%
MCTS bianco (effettua la prima mossa)	100%	0%	0%
MCTS nero (effettua la seconda mossa)	100%	0%	0%

4.2.3 Negamax vs Young Brothers Wait

Young Brothers Wait offre un'ottimizzazione dell'algoritmo Negamax, sfruttando la parallelizzazione.

Per verificare che YBWC implichi un effettivo miglioramento delle prestazioni per il gioco di Hive, all'agente sono stati forniti 4 thread e sono state effettuate 200 partite, considerando un tempo massimo per mossa di un secondo (1s) e alternando quale agente sia il primo giocatore.

Tipologia	Negamax	YBWC	Pareggi
Totale delle partite	91%	4%	5%
Negamax bianco (effettua la prima mossa)	96%	2%	2%
Negamax nero (effettua la seconda mossa)	86%	6%	8%

Il risultato si distacca dalle aspettative. Attribuiamo le cattive performance dell'algoritmo Young Brothers Wait Concept all'overhead dato dal parallelismo e alla forte componente aleatoria che caratterizza il suo comportamento.

4.2.4 Negamax vs MTD(f)

MTD(f) offre un'ottimizzazione dell'algoritmo Negamax, sfruttando la proprietà *zero-window*.

Per verificare che MTD(f) implichi un effettivo miglioramento delle prestazioni per il gioco di Hive, all'agente è stata fornita una tabella di trasposizione di 512 MiB e sono state effettuate 200 partite, considerando un tempo massimo per mossa di un secondo (1s) e alternando quale agente sia il primo giocatore.

Tipologia	Negamax	MTD(f)	Pareggi
Totale delle partite	41%	56,5%	2,5%
Negamax bianco (effettua la prima mossa)	55%	42%	3%
Negamax nero (effettua la seconda mossa)	27%	71%	2%

A livello globale l'algoritmo MTD(f) esibisce prestazioni migliori. Tuttavia, è importante evidenziare quanto sia discriminante il ruolo della prima mossa.

4.2.5 MTD(f) vs MCTS

Dati i precedenti risultati MTD(f) risulta essere l'algoritmo migliore tra gli algoritmi alfa-beta. Abbiamo quindi ritenuto rilevante confrontarlo con Monte Carlo Tree Search, notoriamente un ottimo algoritmo per la risoluzione di giochi da tavolo strategici come Go, Dama e Backgammon.

Per il confronto sono state effettuate 200 partite, considerando un tempo massimo per mossa di un secondo (1s) e alternando quale agente sia il primo giocatore. Le impostazioni dei due agenti rimangono consistenti con quelle definite precedentemente.

Tipologia	MTD(F)	MCTS	Pareggi
Totale delle partite	100%	0%	0%
MCTS bianco (effettua la prima mossa)	100%	0%	0%
MCTS nero (effettua la seconda mossa)	100%	0%	0%

È possibile che l'algoritmo Monte Carlo Tree Search non sia compatibile con il dominio del gioco di Hive.

4.2.6 MTD(f) vs MzingaEngine

Come confronto finale abbiamo deciso di far sfidare il nostro miglior agente, MTD(f), con l'agente della suite Mzinga (MzingaEngine) per capire quanto la nostra implementazione fosse competitiva rispetto ad altre note

Tipologia	MTD(F)	Mzinga	Pareggi
Totale delle partite	97%	1.5%	1.5%
MTD(f) bianco (effettua la prima mossa)	96%	1%	3%
MTD(f) nero (effettua la seconda mossa)	98%	2%	0%

I risultati dimostrano che il nostro agente MTD(f) è competitivo rispetto allo stato dell'arte.

4.3 Implementazione di NNUE

Ci siamo informati su questa tecnologia in quanto riteniamo che possa avere molto potenziale, tuttavia non siamo riusciti ad addestrare una rete neurale NNUE che approssimi la nostra funzione di valutazione dello stato di gioco.

Riportiamo comunque di seguito il lavoro svolto a proposito a fini di documentazione.

4.3.1 Generazione dati sintetici

Per la generazione dei dati sintetici sono state effettuate circa 600000 partite tra due agenti Random. Per ognuna di esse è stato estratto uno stato di gioco casuale, compreso tra la cinquantesima e la cinquecentesima mossa.

Tra le informazioni riportate da uno stato di gioco sono incluse:

- la valutazione della nostra funzione euristica.
- il *side to move*, ovvero a quale dei due giocatori tocca effettuare una mossa.
- le feature viste dalla prospettiva del giocatore 1.
- le feature viste dalla prospettiva del giocatore 2.

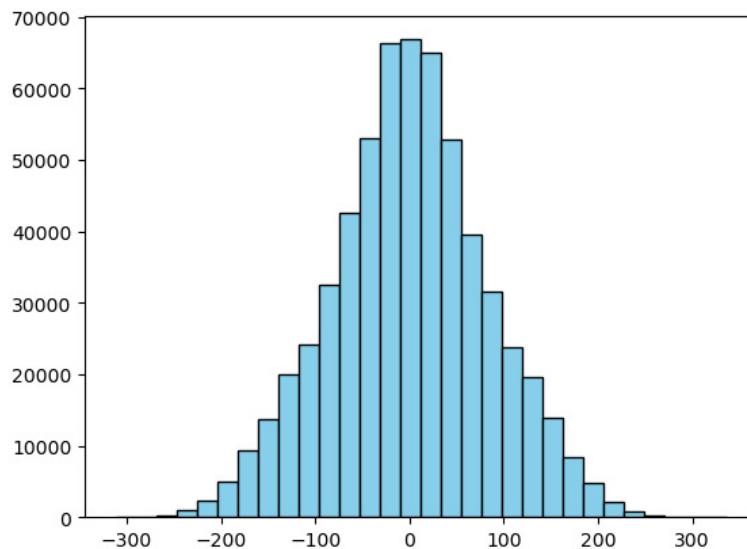


Figura 5: Distribuzione degli score dei nostri dati sintetici

I dati così generati sono stati partizionati randomicamente in tre categorie: **training** costituita dal 60% del totale, **validation** costituita dal 20%, e **testing** costituita dal restante 20%.

4.3.2 Addestramento della rete neurale

Abbiamo tentato di replicare i risultati riportati dagli sviluppatori di *Stockfish* [4].

- L'**architettura** della rete è costituita da due *hidden layer*, composti rispettivamente da 512 e 32 neuroni.
- La **funzione di attivazione** utilizzata, necessaria ad introdurre non linearità nel modello, è *Clipped ReLU*, che ottiene migliori risultati a livello di performance rispetto alla classica *ReLU*. Un ulteriore tentativo è stato utilizzare la funzione *Quantmoid4*, senza però ottenere alcun beneficio.

- La **funzione di loss** è calcolata mediante l'errore quadratico medio tra lo *score* e la predizione del modello. Prima che venga calcolata la loss, entrambi i valori sono divisi per uno scaling factor di 40 e vi è applicata la funzione sigmoide.
- L'**ottimizzatore** adoperato è *Adam*, Adaptive Moment Estimation, poichè riporta risultati vantaggiosi applicato ad altre reti neurali. Il learning rate è impostato a 10^{-5} .
- Il **numero di epoche** è stato impostato a 800.
- La **batch size** è stata impostata a 64 per limitazioni hardware.

Specifiche hardware

- **CPU:** Intel Core i7-14700HX 5.5 GHz
- **Memoria:** 32 GiB di RAM
- **GPU:** NVIDIA GeForce RTX 4070 Max-Q / Mobile

Risultati del training

L'allenamento della rete non è andato a buon fine. Qualsiasi tentativo si è concluso inevitabilmente in un overfitting dei dati di training senza apprendere alcun pattern. Sono stati effettuati vari esperimenti per risolvere il problema ma sono risultati inefficaci. Qui un elenco di alcuni tentativi:

- **Dropout.** Aggiunta di layer di dropout testando con le probabilità tra 0.2 e 0.8.
- **Architettura di rete.** Aggiunta di layer lineari in coda alle rete di dimensione tra 8 e 32 neuroni e potenziamento del primo *hidden layer* a 3072 neuroni.
- **Modifica iperparametri.** Modificati il learning rate (tra 10^{-3} a 10^{-6}), la batch size (tra 64 e 512) e il numero di epoche (tra 100 e 800).

È possibile, data la complessità della funzione euristica che richiede l'utilizzo dell'algoritmo di Tarjan per trovare i punti di articolazione, che sia necessaria un numero di neuroni maggiore per riuscire ad apprendere il metodo di valutazione. Tuttavia una rete più grande invaliderebbe la proprietà di *aggiornamento efficiente* di una NNUE e dunque ingrandirla in modo ingenuo non costituisce una soluzione.

Abbiamo individuato tre possibili strade per migliorare le prestazioni del modello:

- Semplificazione della funzione euristica con possibile rimozione dell'algoritmo di Tarjan.
- Aggiunta features in ingresso, per facilitare la comprensione dello stato di gioco.
- Individuare la dimensione di rete minima che riesce ad apprendere e valutarne le performance.

Riferimenti bibliografici

- [1] Duncan Kampert et al. «Mimicking the Human Approach in the Game of Hive». In: *2021 IEEE Symposium Series on Computational Intelligence (SSCI)*. 2021, pp. 1–8. DOI: 10.1109/SSCI50451.2021.9659999.
- [2] Jon Thysell. *Mzinga: open-source software to play the board game Hive*. URL: <https://github.com/jonthysell/Mzinga>.
- [3] Rainer Feldmann et al. «Distributed Game-Tree Search». In: *ICGA Journal* 12 (1989), pp. 65–73. URL: <https://api.semanticscholar.org/CorpusID:46859244>.
- [4] Tomasz Sobczyk. *NNUE*. URL: <https://official-stockfish.github.io/docs/nnue-pytorch-wiki/docs/nnue.html>.